

Rapport Technique : Déploiement d'une Architecture Haute Disponibilité avec HAProxy et Nginx -Nathaniel T

1. Contexte du Projet (M2L) et Objectifs

Ce rapport s'inscrit dans le cadre de la refonte de l'infrastructure informatique de la **M2L (Maison des Liges de Lorraine)**. Face à la croissance constante des besoins numériques des différentes ligues sportives hébergées et pour garantir un accès ininterrompu à leurs services web (extranet, applications métiers, portails d'information), la mise en place d'une architecture résiliente est devenue indispensable.

L'objectif de ce projet est de concevoir, déployer et sécuriser une infrastructure web hautement disponible au sein de la zone démilitarisée (DMZ) de la M2L. L'architecture repose sur un répartiteur de charge (Load Balancer) HAProxy qui distribue le trafic entrant de manière transparente vers deux serveurs web Nginx, permettant ainsi d'absorber les pics de charge (lors des événements sportifs) et d'assurer une tolérance aux pannes.

Le projet inclut également la sécurisation des échanges (TLS/SSL), la persistance des sessions utilisateurs, le filtrage réseau, la supervision des services et l'initiation à l'automatisation des déploiements pour faciliter la maintenance par le service informatique de la M2L.

2. Architecture Réseau (DMZ)

L'infrastructure est déployée sur le plan d'adressage suivant défini pour la DMZ de la M2L :

- **Passerelle (Gateway) :** 192.168.120.1
- **Masque de sous-réseau :** 255.255.255.240 (/28)
- **Serveur DNS :** 192.168.110.10

- **Serveur Répartiteur (HAProxy) :** 192.168.120.3
- **Serveur Web 1 (Nginx) :** 192.168.120.4
- **Serveur Web 2 (Nginx) :** 192.168.120.5

3. Phase 1 : Déploiement des Serveurs Web (Nginx)

3.1. Résolution de conflit de port (Nettoyage système)

Lors de l'installation initiale, un conflit sur le port TCP 80 a été détecté (erreur `bind() to 0.0.0.0:80 failed (98: Address already in use)`). Le service Apache2 préinstallé monopolisait le port.

Une purge complète a été effectuée sur les serveurs web :

```
sudo systemctl stop apache2
sudo apt purge apache2 apache2-utils apache2-bin apache2.2-common -y
sudo apt autoremove -y
```

3.2. Installation et personnalisation de Nginx

Sur les deux serveurs (`192.168.120.4` et `192.168.120.5`), Nginx a été installé et configuré :

```
sudo apt update && sudo apt install nginx -y
sudo systemctl enable --now nginx
```

Pour identifier les serveurs lors des tests de répartition, la page d'accueil par défaut a été modifiée :

- **Sur Web1 :** `echo "<h1>Bienvenue sur le Serveur Web 1 (192.168.120.4) - M2L</h1>" | sudo tee /var/www/html/index.nginx-debian.html`
- **Sur Web2 :** `echo "<h1>Bienvenue sur le Serveur Web 2 (192.168.120.5) - M2L</h1>" | sudo tee /var/www/html/index.nginx-debian.html`

4. Phase 2 : Déploiement du Répartiteur de Charge (HAProxy)

HAProxy a été installé sur le serveur frontal `192.168.120.3` :

```
sudo apt install haproxy -y
```

4.1. Configuration de base et gestion des Timeouts

Le fichier `/etc/haproxy/haproxy.cfg` a été configuré. Pour éviter les avertissements (`warnings`) liés à l'absence de délais d'expiration, des `timeouts` ont été définis dans la section `defaults`.

```
defaults
    mode      http
    timeout connect 5000ms
    timeout client 50000ms
    timeout server 50000ms
```

4.2. Sécurisation TLS (HTTPS) et Sticky Sessions

Un certificat auto-signé a été généré pour chiffrer les communications (simulant un futur certificat officiel pour la M2L) ou sinon dans mon cas utilisé un certificat de notre autorité de certification:

```
sudo mkdir -p /etc/ssl/haproxy
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/haproxy/haproxy.key -out /etc/ssl/haproxy/haproxy.crt
sudo cat /etc/ssl/haproxy/haproxy.crt /etc/ssl/haproxy/haproxy.key | sudo tee /etc/ssl/haproxy/haproxy.pem
```

La configuration des `frontend` et `backend` a été mise à jour pour inclure la redirection HTTPS et l'insertion de cookies pour la persistance de session (Sticky Sessions), essentielle pour les applications métiers des ligues :

```
frontend http_front
    bind *:80
    # Redirection HTTP vers HTTPS
    http-request redirect scheme https unless { ssl_fc }

    # Écoute sur le port 443 avec le certificat TLS
    bind *:443 ssl crt /etc/ssl/haproxy/haproxy.pem
    default_backend http_back

backend http_back
    balance roundrobin
    # Configuration des Sticky Sessions
    cookie SERVERID insert indirect nocache

    server web1 192.168.120.4:80 cookie web1 check
    server web2 192.168.120.5:80 cookie web2 check
```

5. Phase 3 : Durcissement et Supervision

5.1. Tableau de bord des statistiques HAProxy

Une interface de supervision a été activée pour monitorer l'état du cluster en temps réel (accessible via le port 8404), permettant à l'équipe IT de la M2L de réagir proactivement.

```
listen stats
  bind *:8404
  stats enable
  stats uri /stats
  stats hide-version
  stats auth admin:SuperMotDePasse
```

5.2. Health Checks de Niveau 7 (Applicatif)

Pour éviter qu'HAProxy n'envoie du trafic vers un serveur dont le service web est planté mais le port 80 toujours ouvert, une vérification sur un fichier spécifique a été mise en place.

- **Sur Web1 et Web2 :** `echo "OK" | sudo tee /var/www/html/health.html`
- **Dans HAProxy (backend) :** Ajout de la directive `option httpchk GET /health.html`

5.3. Filtrage Réseau avec UFW

Pour empêcher l'accès direct aux serveurs web (contournement du Load Balancer), des règles de pare-feu strictes ont été appliquées sur Web1 et Web2 :

```
sudo apt install ufw -y
sudo ufw allow ssh
sudo ufw allow from 192.168.120.3 to any port 80
sudo ufw enable
```

6. Phase 4 : Automatisation du Déploiement

Afin de faciliter la scalabilité horizontale (ajout rapide de nouveaux serveurs web en cas de fort trafic imprévu), un script d'automatisation Bash

`install_web.sh` a été développé :

```
#!/bin/bash
# Script de déploiement automatisé d'un nœud web Nginx pour la M2L

echo "[*] Mise à jour et installation de Nginx..."
apt update -y && apt install nginx -y

echo "[*] Création de la page par défaut..."
echo "<h1>Bienvenue sur le nouveau serveur Web (Déployé automatiquement)</h1>" > /var/
www/html/index.nginx-debian.html
echo "OK" > /var/www/html/health.html

echo "[*] Démarrage et activation du service..."
systemctl enable --now nginx

echo "[*] Installation terminée avec succès. Nœud prêt à être ajouté au HAProxy."
```

Utilisation : `chmod +x install_web.sh` puis `sudo ./install_web.sh`.

7. Plan de Validation et de Tests

Afin de s'assurer du bon fonctionnement de l'infrastructure avant sa mise en production pour les ligues, les tests suivants doivent être validés :

1. **Test de la répartition de charge (Round Robin) :** - *Action* : Effectuer plusieurs requêtes HTTP depuis une machine cliente (`curl -k https://192.168.120.3`).
 - *Résultat attendu* : La réponse doit alterner entre Web1 et Web2 si la session n'est pas persistante (sans cookie).
2. **Test de persistance de session (Sticky Sessions) :**
 - *Action* : Se connecter via un navigateur web classique. Actualiser la page plusieurs fois.
 - *Résultat attendu* : L'utilisateur reste sur le même serveur (Web1 ou Web2) durant toute la session grâce au cookie `SERVERID`.
3. **Test de bascule en cas de panne (Failover / Health Check) :**
 - *Action* : Arrêter le service Nginx sur Web1 (`sudo systemctl stop nginx`) ou supprimer le fichier `health.html`.
 - *Résultat attendu* : HAProxy détecte la panne et redirige 100% du trafic vers Web2. La page de statistiques indique Web1 en statut "DOWN".
4. **Test de sécurité (Filtrage UFW) :**

- *Action* : Tenter d'accéder directement à `http://192.168.120.4` depuis une machine cliente (autre que HAProxy).
- *Résultat attendu* : La connexion doit être refusée (Timeout).

8. Conclusion

L'architecture finale répond parfaitement aux exigences de la Maison des Ligues de Lorraine : haute disponibilité, sécurité de base et scalabilité. Le trafic est chiffré, réparti efficacement, les sessions utilisateurs sont maintenues pour les applications métiers, et les accès directs aux backends sont bloqués. Cette infrastructure constitue une excellente base, robuste, qui pourra par la suite être gérée de manière totalement centralisée via des outils de type IaC (Infrastructure as Code) comme Ansible.